Case study 1

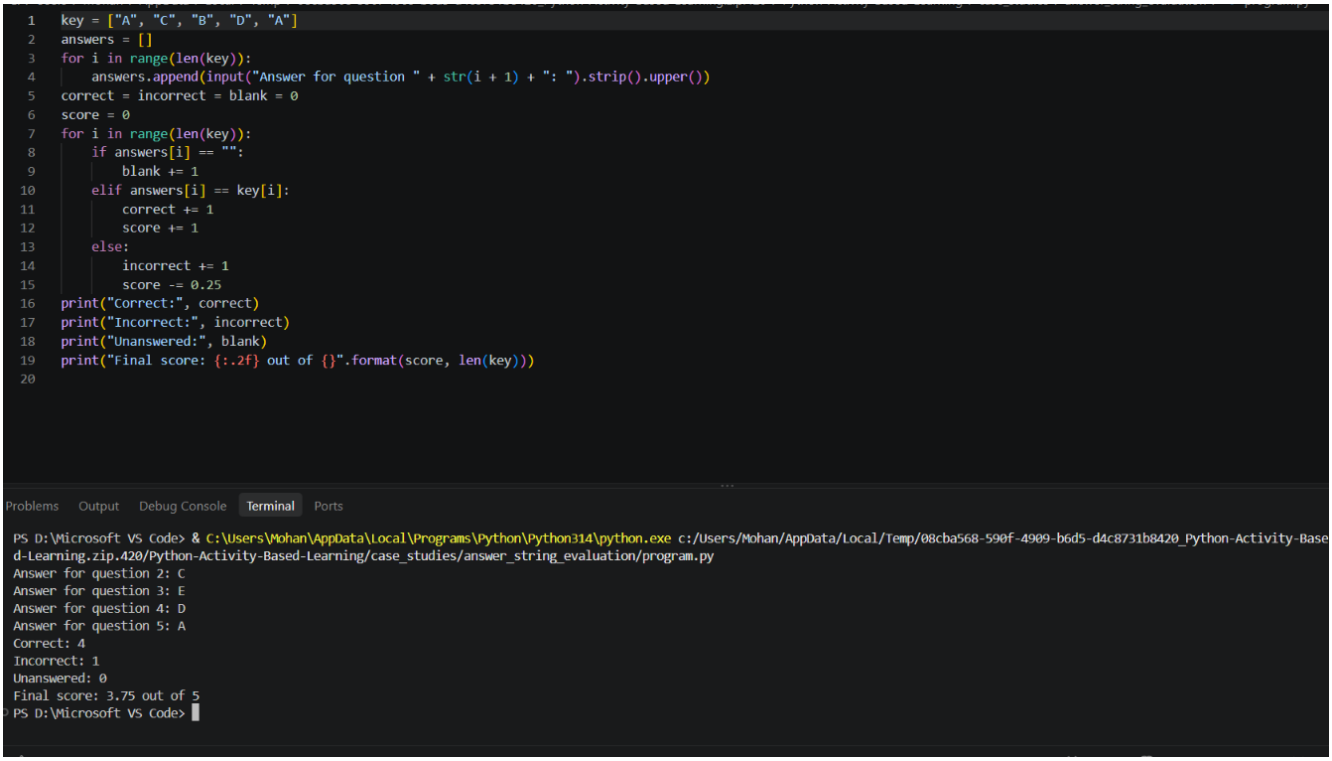
1. aim: String-Based Examination Answer Evaluation:

2. objectives: Compare answer strings against an answer key, calculate correct, incorrect, and unanswered responses, apply negative marking, and produce student-wise results.

3. python code:

```
key = ["A", "C", "B", "D", "A"]
answers = []
for i in range(len(key)):
    answers.append(input("Answer for question " + str(i + 1) + ": ").strip().upper())
correct = incorrect = blank = 0
score = 0
for i in range(len(key)):
    if answers[i] == "":
        blank += 1
    elif answers[i] == key[i]:
        correct += 1
        score += 1
    else:
        incorrect += 1
        score -= 0.25
print("Correct:", correct)
print("Incorrect:", incorrect)
print("Unanswered:", blank)
print("Final score: {:.2f} out of {}".format(score, len(key)))
```

4. screenshots:



Case study 2

1. aim: ATM Transaction Sequence and Balance Validation

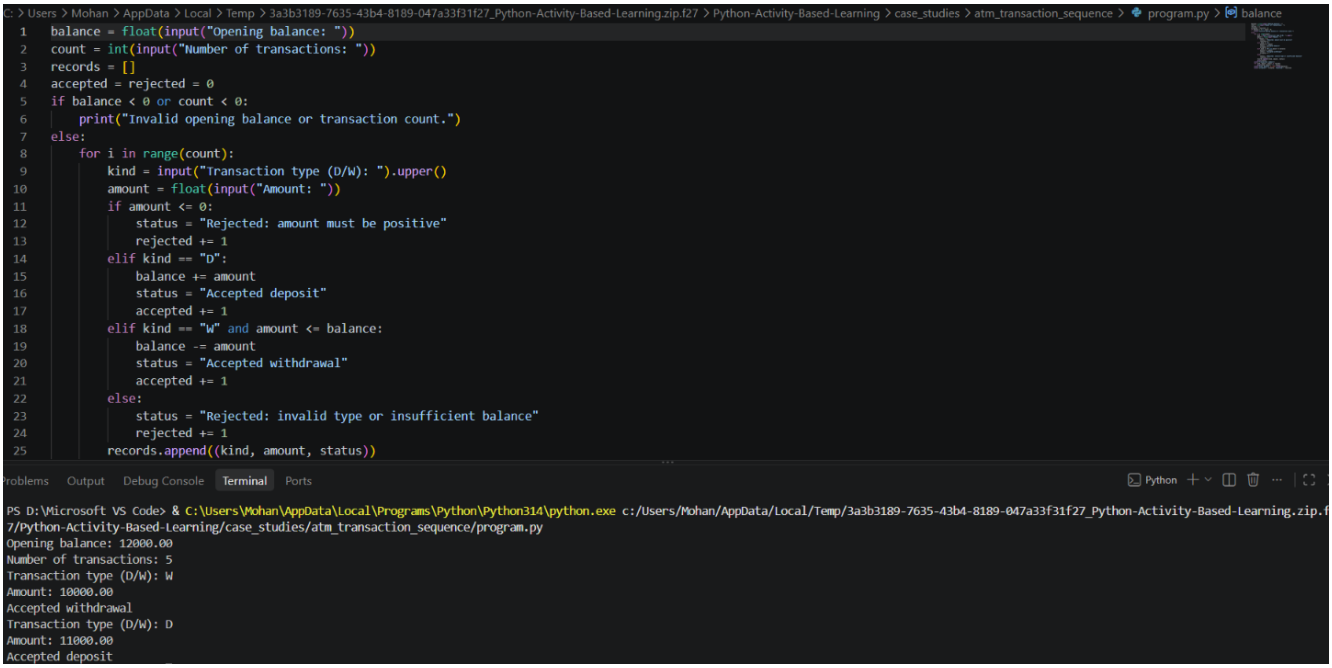
2. objectives: Process a sequence of deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary**.**

3. python code:

```
balance = float(input("Opening balance: "))
count = int(input("Number of transactions: "))
records = []
accepted = rejected = 0
if balance < 0 or count < 0:
    print("Invalid opening balance or transaction count.")
else:
    for i in range(count):
        kind = input("Transaction type (D/W): ").upper()
        amount = float(input("Amount: "))
        if amount <= 0:
            status = "Rejected: amount must be positive"
            rejected += 1
        elif kind == "D":
            balance += amount
            status = "Accepted deposit"
            accepted += 1
        elif kind == "W" and amount <= balance:
            balance -= amount
            status = "Accepted withdrawal"
```

```
            accepted += 1
        else:
            status = "Rejected: invalid type or insufficient balance"
            rejected += 1
        records.append((kind, amount, status))
        print(status)
    print("Transaction summary")
    for kind, amount, status in records:
        print(kind, amount, "-", status)
    print("Final balance: {:.2f}".format(balance))
    print("Accepted:", accepted, "Rejected:", rejected)
```

4. screenshots:



Case study 3

1. aim: Academic Attendance Recovery and Target Tracker

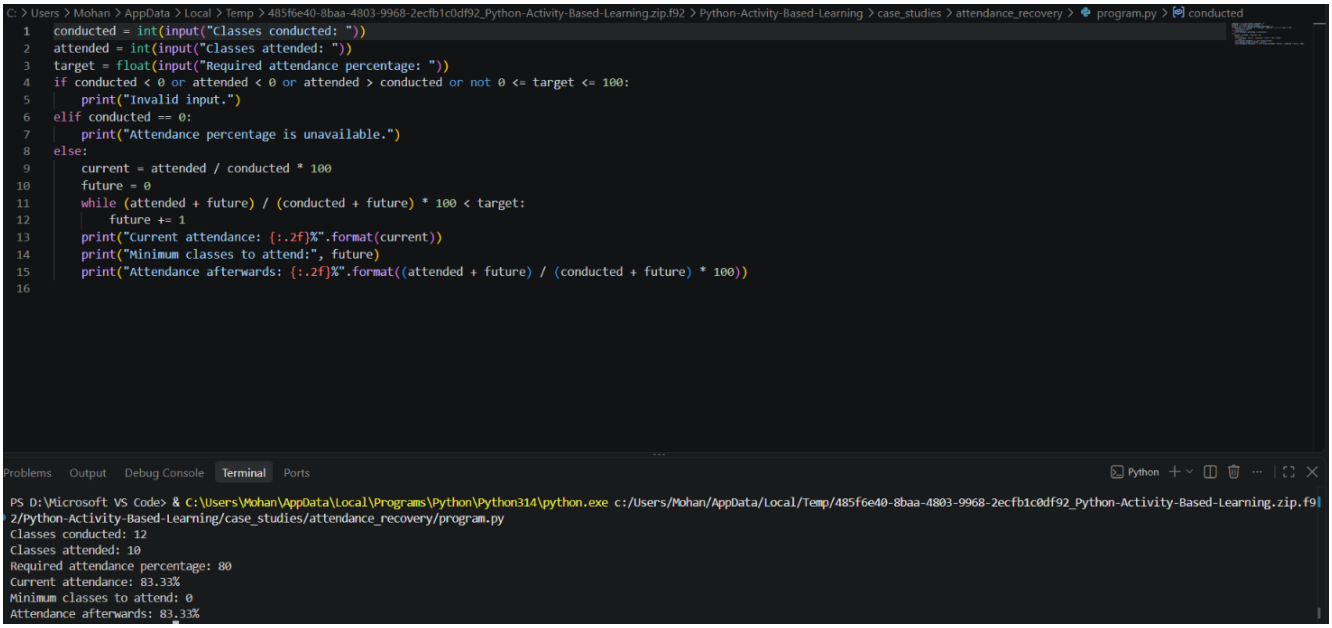
2. objectives: Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold under different future attendance scenarios.

3. python code:

```
conducted = int(input("Classes conducted: "))
attended = int(input("Classes attended: "))
target = float(input("Required attendance percentage: "))
if conducted < 0 or attended < 0 or attended > conducted or not 0 <= target <= 100:
    print("Invalid input.")
elif conducted == 0:
    print("Attendance percentage is unavailable.")
else:
```

```
    current = attended / conducted * 100
    future = 0
    while (attended + future) / (conducted + future) * 100 < target:
        future += 1
    print("Current attendance: {:.2f}%".format(current))
    print("Minimum classes to attend:", future)
    print("Attendance afterwards: {:.2f}%".format((attended + future) / (conducted + future)
* 100))
```

4. screenshots:



Case study 4

1. aim: Automated Examination Seating Arrangement Generator

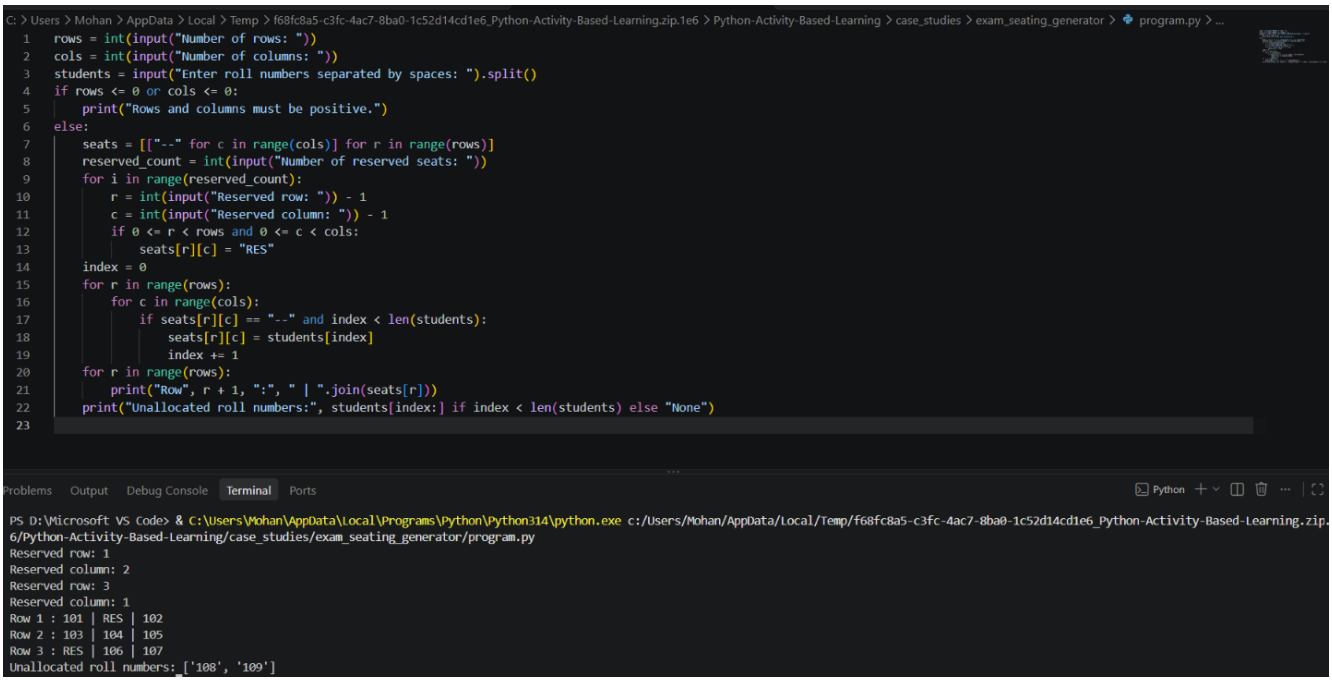
2. objectives: Generate a seating arrangement with row and column identifiers, reserved positions, alternate seating restrictions, and fixed classroom capacities.

3. python code:

```
rows = int(input("Number of rows: "))
cols = int(input("Number of columns: "))
students = input("Enter roll numbers separated by spaces: ").split()
if rows <= 0 or cols <= 0:
    print("Rows and columns must be positive.")
else:
    seats = [["--" for c in range(cols)] for r in range(rows)]
    reserved_count = int(input("Number of reserved seats: "))
    for i in range(reserved_count):
        r = int(input("Reserved row: ")) - 1
        c = int(input("Reserved column: ")) - 1
        if 0 <= r < rows and 0 <= c < cols:
```

```
            seats[r][c] = "RES"
    index = 0
    for r in range(rows):
        for c in range(cols):
            if seats[r][c] == "--" and index < len(students):
                seats[r][c] = students[index]
                index += 1
    for r in range(rows):
        print("Row", r + 1, ":", " | ".join(seats[r]))
    print("Unallocated roll numbers:", students[index:] if index < len(students) else "None")
```

4. screenshots:



Case study 5

1. aim: Food Waste Monitoring and Efficiency Metrics

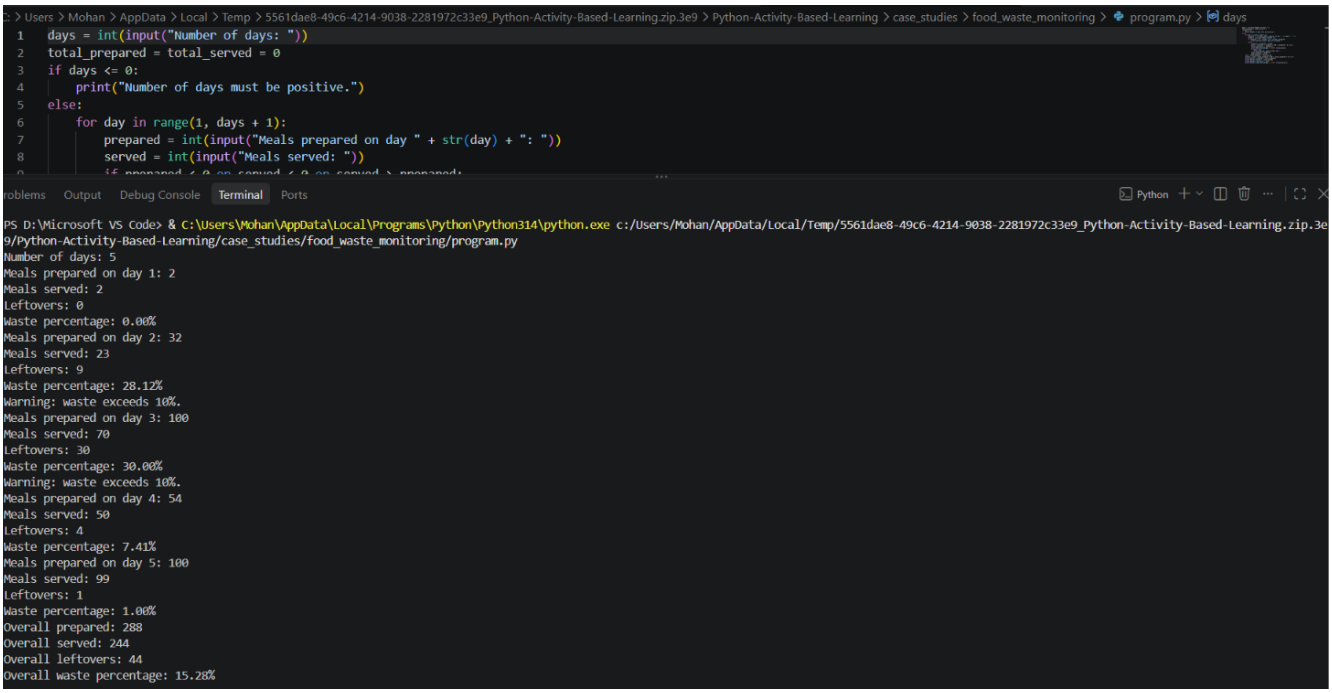
2. objectives: Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptable limits**.**

3. python code:

```
days = int(input("Number of days: "))
total_prepared = total_served = 0
if days <= 0:
    print("Number of days must be positive.")
else:
    for day in range(1, days + 1):
        prepared = int(input("Meals prepared on day " + str(day) + ": "))
        served = int(input("Meals served: "))
        if prepared < 0 or served < 0 or served > prepared:
```

```
            print("Invalid counts; day not included.")
        else:
            leftovers = prepared - served
            waste = leftovers / prepared * 100 if prepared > 0 else 0
            print("Leftovers:", leftovers)
            print("Waste percentage: {:.2f}%".format(waste))
            if waste > 10:
                print("Warning: waste exceeds 10%.")
            total_prepared += prepared
            total_served += served
    left = total_prepared - total_served
    overall = left / total_prepared * 100 if total_prepared > 0 else 0
    print("Overall prepared:", total_prepared)
    print("Overall served:", total_served)
    print("Overall leftovers:", left)
    print("Overall waste percentage: {:.2f}%".format(overall))
```

4. screenshots:



Case study 6

1. aim: Retail Shopping Budget Analysis and Discount Calculator

2. objectives: Evaluate a shopping basket containing multiple products and quantities, apply discount conditions, and determine which purchases must be reduced to remain within a fixed budget.

3. python code:

```
names = []
totals = []
```

```
n = int(input("Number of products: "))
budget = float(input("Shopping budget (₹): "))
subtotal = 0
if n < 0 or budget < 0:
    print("Invalid input.")
else:
    for i in range(n):
        name = input("Product name: ")
        price = float(input("Unit price (₹): "))
        quantity = int(input("Quantity: "))
        if price < 0 or quantity < 0:
            print("Invalid price or quantity.")
            break
        names.append(name)
        totals.append(price * quantity)
        subtotal += price * quantity
    else:
        discount = subtotal * 0.10 if subtotal >= 2000 else 0
        final = subtotal - discount
        for i in range(len(names)):
            print(names[i], ": ₹{:.2f}".format(totals[i]))
        print("Subtotal: ₹{:.2f}".format(subtotal))
        print("Discount: ₹{:.2f}".format(discount))
        print("Final total: ₹{:.2f}".format(final))
        if final > budget:
            print("Over budget by: ₹{:.2f}".format(final - budget))
        else:
            print("Within budget; remaining: ₹{:.2f}".format(budget - final))
```

4. screenshots:

C: > Users > Mohan > AppData > Local > Temp > 8141413e-e3e7-4820-abac-c70b8fb9e54e_Py

```python
names = []
totals = []
n = int(input("Number of products: "))
budget = float(input("Shopping budget (₹): "))
subtotal = 0
if n < 0 or budget < 0:
    print("Invalid input.")
else:
    for i in range(n):
        name = input("Product name: ")
        price = float(input("Unit price (₹): "))
        quantity = int(input("Quantity: "))
        if price < 0 or quantity < 0:
            print("Invalid price or quantity.")
            break
        names.append(name)
        totals.append(price * quantity)
        subtotal += price * quantity
    else:
        discount = subtotal * 0.10 if subtotal >= 2000 else 0
        final = subtotal - discount
```

roblems Output Debug Console Terminal Ports

```
PS D:\Microsoft VS Code> & C:\Users\Mohan\AppData\Local\Programs\Python\Pyth
e/Python-Activity-Based-Learning/case_studies/shopping_budget/program.py
Product name: television
Unit price (₹): 24000.00
Quantity: 1
Product name: air conditioner
Unit price (₹): 25000.00
Quantity: 1
television : ₹24000.00
air conditioner : ₹25000.00
Subtotal: ₹49000.00
Discount: ₹4900.00
Final total: ₹44100.00
Within budget; remaining: ₹5900.00
```

Case study 7

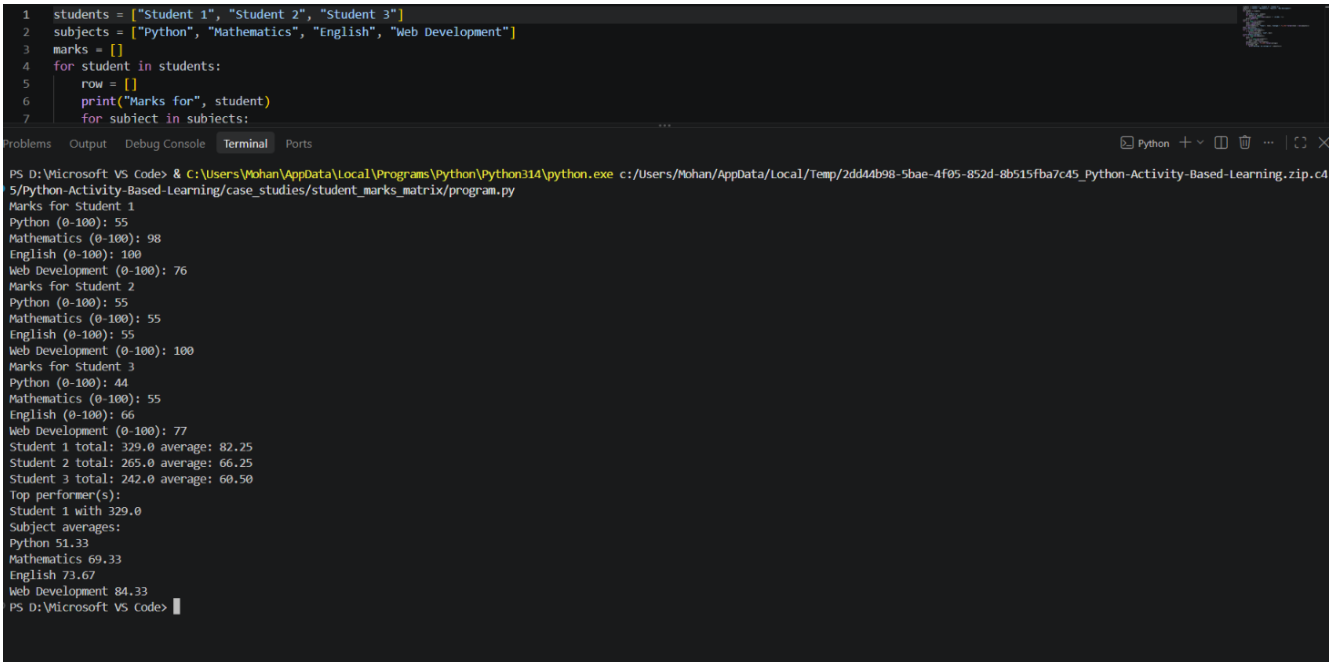
1. aim: Student Performance Matrix and Subject Analysis

2. objectives: Analyse a matrix of student marks across subjects, calculate row-wise and column-wise statistics, identify top performers, and detect subjects with low average performance**.**

3. python code:

```
students = ["Student 1", "Student 2", "Student 3"]
subjects = ["Python", "Mathematics", "English", "Web Development"]
marks = []
for student in students:
    row = []
    print("Marks for", student)
    for subject in subjects:
        row.append(float(input(subject + " (0-100): ")))
    marks.append(row)
totals = []
for i in range(len(students)):
    total = sum(marks[i])
    totals.append(total)
    print(students[i], "total:", total, "average:", "{:.2f}".format(total / len(subjects)))
best = max(totals)
print("Top performer(s):")
for i in range(len(students)):
    if totals[i] == best:
        print(students[i], "with", best)
print("Subject averages:")
for c in range(len(subjects)):
    total = 0
    for r in range(len(students)):
        total += marks[r][c]
    average = total / len(students)
    print(subjects[c], "{:.2f}".format(average))
    if average < 50:
        print("Warning: low average in", subjects[c])
```

4. screenshots:



Case study 8

1. aim: Student Exam Performance Trend and Weakness Identifier

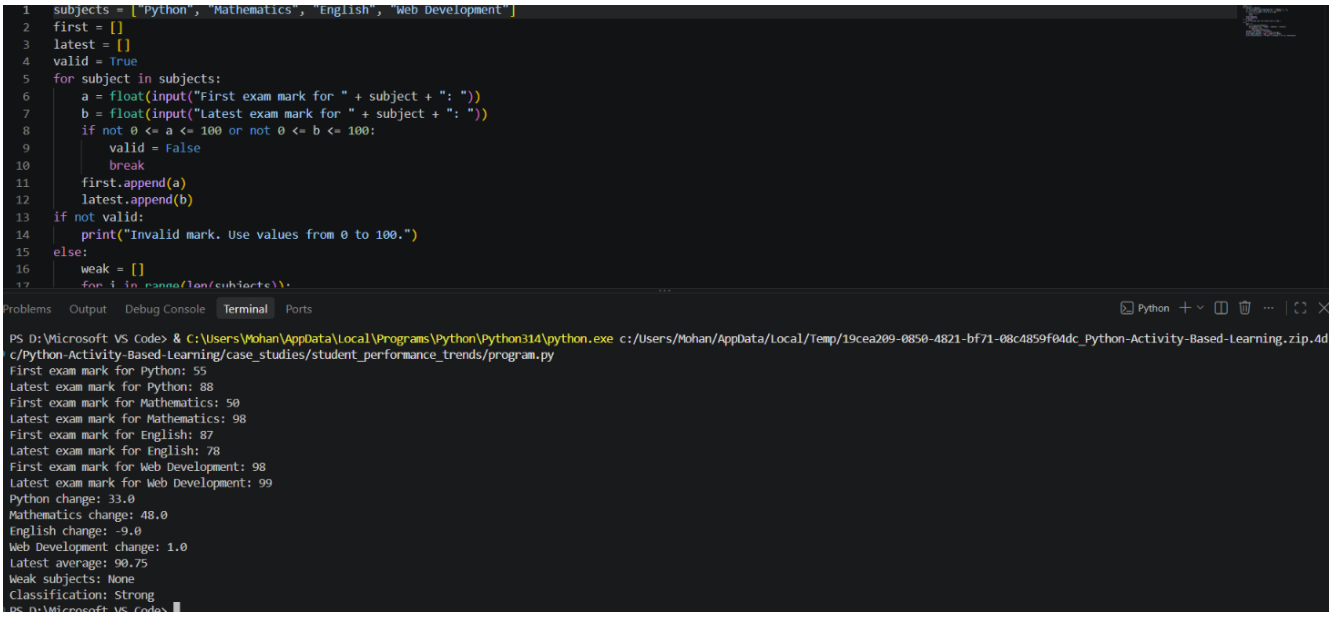
2. objectives: Compare assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and classify students according to defined criteria.

3. python code:

```
subjects = ["Python", "Mathematics", "English", "Web Development"]
first = []
latest = []
valid = True
for subject in subjects:
    a = float(input("First exam mark for " + subject + ": "))
    b = float(input("Latest exam mark for " + subject + ": "))
    if not 0 <= a <= 100 or not 0 <= b <= 100:
        valid = False
        break
    first.append(a)
    latest.append(b)
if not valid:
    print("Invalid mark. Use values from 0 to 100.")
else:
    weak = []
    for i in range(len(subjects)):
        print(subjects[i], "change:", latest[i] - first[i])
        if latest[i] < 40:
            weak.append(subjects[i])
    average = sum(latest) / len(latest)
    print("Latest average: {:.2f}".format(average))
```

```
    print("Weak subjects:", weak if weak else "None")
    print("Classification:", "Strong" if average >= 75 else "Developing")
```

4. screenshots:



Case study 9

1. aim: Run-Length Text Compression and Analysis System

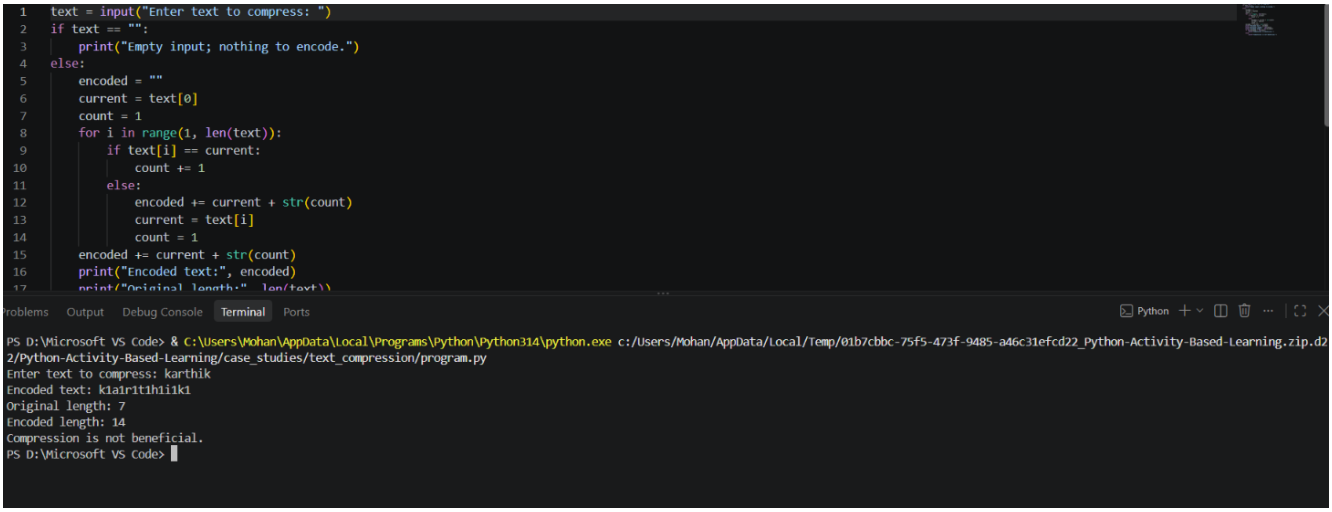
2. objectives: Encode a string by grouping consecutive repeated characters and compare original and encoded lengths, including cases where encoding is not beneficial**.**

3. python code:

```
text = input("Enter text to compress: ")
if text == "":
    print("Empty input; nothing to encode.")
else:
    encoded = ""
    current = text[0]
    count = 1
    for i in range(1, len(text)):
        if text[i] == current:
            count += 1
        else:
            encoded += current + str(count)
            current = text[i]
            count = 1
    encoded += current + str(count)
    print("Encoded text:", encoded)
    print("Original length:", len(text))
    print("Encoded length:", len(encoded))
    if len(encoded) < len(text):
```

```
        print("Compression is beneficial.")
    else:
        print("Compression is not beneficial.")
```

4. screenshots:



Case study 10

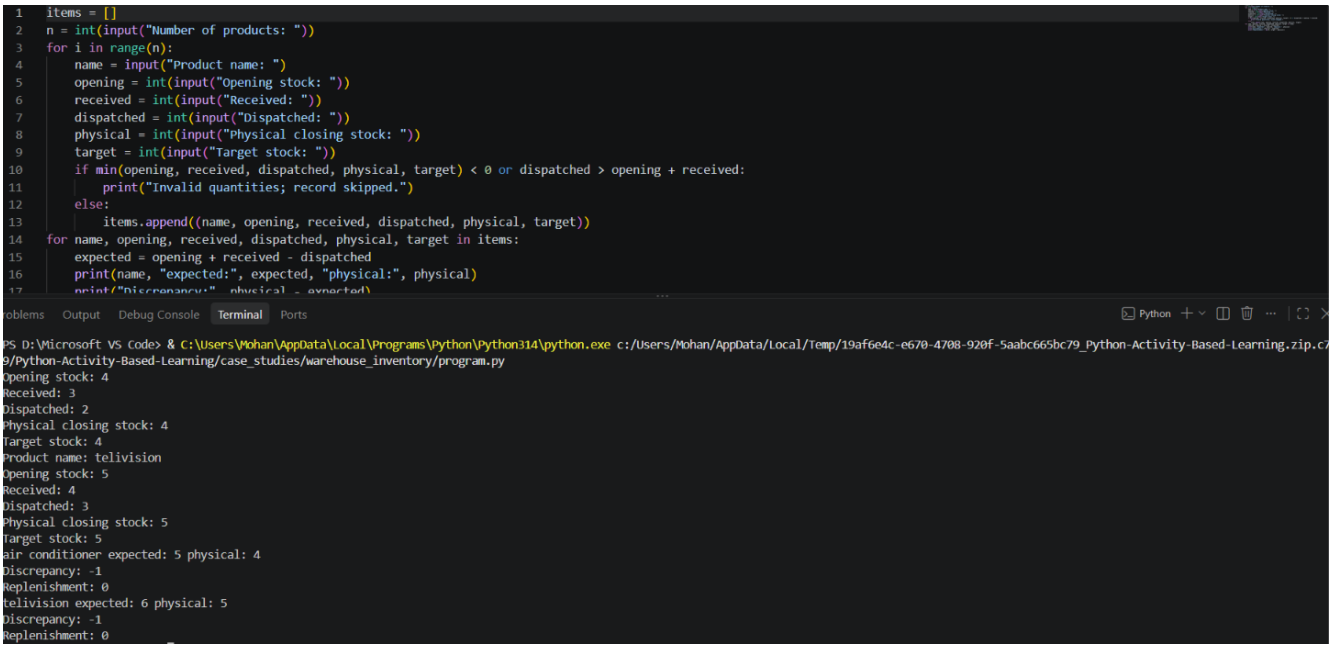
1. aim: Warehouse Inventory Reconciliation and Order Planning

2. objectives: Compare opening stock, incoming quantities, dispatched quantities, and closing stock across multiple products; detect discrepancies and calculate replenishment requirements.

3. python code:

```
items = []
n = int(input("Number of products: "))
for i in range(n):
    name = input("Product name: ")
    opening = int(input("Opening stock: "))
    received = int(input("Received: "))
    dispatched = int(input("Dispatched: "))
    physical = int(input("Physical closing stock: "))
    target = int(input("Target stock: "))
    if min(opening, received, dispatched, physical, target) < 0 or dispatched > opening +
received:
        print("Invalid quantities; record skipped.")
    else:
        items.append((name, opening, received, dispatched, physical, target))
for name, opening, received, dispatched, physical, target in items:
    expected = opening + received - dispatched
    print(name, "expected:", expected, "physical:", physical)
    print("Discrepancy:", physical - expected)
    print("Replenishment:", max(0, target - physical))
```

4. screenshots:



```
items = []
n = int(input("Number of products: "))
for i in range(n):
    name = input("Product name: ")
    opening = int(input("Opening stock: "))
    received = int(input("Received: "))
    dispatched = int(input("Dispatched: "))
    physical = int(input("Physical closing stock: "))
    target = int(input("Target stock: "))
    if min(opening, received, dispatched, physical, target) < 0 or dispatched > opening + received:
        print("Invalid quantities; record skipped.")
    else:
        items.append((name, opening, received, dispatched, physical, target))
for name, opening, received, dispatched, physical, target in items:
    expected = opening + received - dispatched
    print(name, "expected:", expected, "physical:", physical)
    print("Discrepancy:", physical - expected)
```

```
roblems  Output  Debug Console  Terminal  Ports                                                    Python

PS D:\Microsoft VS Code> & C:\Users\Mohan\AppData\Local\Programs\Python\Python314\python.exe c:/Users/Mohan/AppData/Local/Temp/19af6e4c-e670-4708-920f-5aabc665bc79_Python-Activity-Based-Learning.zip.c7
9/Python-Activity-Based-Learning/case_studies/warehouse_inventory/program.py
Opening stock: 4
Received: 3
Dispatched: 2
Physical closing stock: 4
Target stock: 4
Product name: telivision
Opening stock: 5
Received: 4
Dispatched: 3
Physical closing stock: 5
Target stock: 5
air conditioner expected: 5 physical: 4
Discrepancy: -1
Replenishment: 0
telivision expected: 6 physical: 5
Discrepancy: -1
Replenishment: 0
```